CAHIER DES CHARGES

Conception et simulation sous ROS 2 d'un système intelligent de contrôle d'accès à un parking

· ROS 2 Jazzy · Gazebo Harmonic · Python

> **Détecter** le véhicule, **décider** de l'accès, **commander** la barrière : le tout en simulation avec ROS 2 et Gazebo.

1. Objectif

Simuler sous ROS 2 et Gazebo une barrière de parking qui s'ouvre pour un véhicule autorisé, reste fermée sinon, et rouvre si un obstacle est détecté pendant la fermeture. Tout se fait en simulation, sans matériel.

2. Règles de fonctionnement

1. Sans véhicule, la barrière reste fermée.
2. Véhicule détecté : le système attend une autorisation (simulée par une commande logicielle).
3. Accès autorisé : la barrière s'ouvre (rotation de 0° à 90°). Accès refusé : elle reste fermée.
4. Tant que la zone de passage est occupée, la barrière reste ouverte.
5. Zone libre pendant 2 s : la barrière se ferme.
6. Obstacle pendant la fermeture : elle s'arrête et se rouvre.

Le délai de 2 s est un paramètre ROS 2 modifiable. Sans autorisation après 10 s, la demande expire et la barrière reste fermée.

3. Outils

Ubuntu 24.04, ROS 2 Jazzy, Gazebo Harmonic, Python (`rclpy`), modèles SDF, `ros_gz_bridge` pour relier ROS 2 et Gazebo. RViz 2 est facultatif.

4. Architecture

Trois nœuds Python communiquent par topics, avec Gazebo pour la simulation :

- **`sensor_node`** : lit les capteurs simulés (véhicule à l'entrée, présence dans la zone de passage).
- **`controller_node`** : machine à états qui décide d'ouvrir, de maintenir ou de fermer.
- **`barrier_node`** : envoie l'angle voulu à l'articulation de la barrière dans Gazebo.

Topic	Type	Rôle
`/parking/vehicle_detected`	`std_msgs/Bool`	Véhicule devant la barrière
`/parking/zone_occupied`	`std_msgs/Bool`	Véhicule ou obstacle dans la zone de passage
`/parking/access_granted`	`std_msgs/Bool`	Autorisation (true) ou refus (false)
`/parking/barrier_command`	`std_msgs/String`	OPEN ou CLOSE
`/parking/barrier_state`	`std_msgs/String`	État affiché dans le terminal

5. États de la barrière

État	Passe à	Condition
FERMÉE	OUVERTURE	Véhicule détecté et accès autorisé
OUVERTURE	OUVERTE	Angle de 90° atteint
OUVERTE	FERMETURE	Zone libre pendant 2 s
FERMETURE	FERMÉE	Angle de 0° atteint
FERMETURE	OUVERTURE	Zone occupée (obstacle)

6. Tests de validation

Test	Résultat attendu
Aucun véhicule	La barrière reste fermée
Véhicule et accès autorisé	La barrière s'ouvre
Véhicule et accès refusé	La barrière reste fermée
Véhicule en train de passer	La barrière reste ouverte

Test	Résultat attendu
Zone libre pendant 2 s	La barrière se ferme
Obstacle pendant la fermeture	La barrière se rouvre

Chaque test doit donner le même résultat à chaque exécution, avec un script ou un launch file qui rejoue le scénario.

7. Plan de réalisation

1. Installer et vérifier ROS 2 Jazzy et Gazebo Harmonic.
2. Créer le workspace et le package ROS 2.
3. Modéliser le parking, la barrière, le véhicule et les capteurs dans Gazebo.
4. Relier Gazebo et ROS 2 avec `ros_gz_bridge`.
5. Programmer les trois nœuds.
6. Tester les six scénarios, corriger, puis préparer la démonstration.

8. Livrables

Le package ROS 2 avec les modèles et le monde Gazebo, un launch file, les scénarios de test, un court rapport technique (architecture, topics, utilisation) et une démonstration.

9. Contraintes

- Utiliser uniquement des outils libres.
- Travailler uniquement en simulation, sans matériel électronique.
- Garder une complexité adaptée à un projet universitaire.
- Permettre de rejouer chaque scénario de test à l'identique.

10. Conclusion

Ce projet met en pratique la robotique et ROS 2 à travers un système de contrôle d'accès simulé : modélisation d'un mécanisme mobile, communication entre nœuds, détection simulée et commande d'un actionneur dans un environnement 3D.